```
1. INITIALIZATION:
   Set initial slope (m) = 0
   Set initial intercept (b) = 0
   Set learning rate (alpha) = 0.01
   Set number of iterations = 1000

2. TRAINING LOOP:
   FOR iteration FROM 1 TO iterations:

     a. MAKE PREDICTIONS:
        FOR each data point (x_i, y_i):
           y_pred[i] = (m * x_i) + b

     b. CALCULATE ERRORS (Loss):
        FOR each prediction:
           error[i] = y_pred[i] - y_i

     c. CALCULATE GRADIENTS (Direction to reduce error):
        m_gradient = (2 / N) * SUM(error[i] * x_i)
        b_gradient = (2 / N) * SUM(error[i])

     d. UPDATE PARAMETERS:
        m = m - (alpha * m_gradient)
        b = b - (alpha * b_gradient)

3. PREDICT NEW DATA:
   FOR any new feature X_new:
      Y_new = (m * X_new) + b
   RETURN Y_new
```

```
1. INITIALIZATION:
   Set initial feature weights (w_1, w_2, ..., w_p) = 0
   Set initial intercept (b) = 0
   Set regularization strength (lambda) = 0.5
   Set learning rate (alpha) = 0.01
   Set number of iterations = 1000

2. TRAINING LOOP:
   FOR iteration FROM 1 TO iterations:

     a. MAKE PREDICTIONS:
        FOR each data point i:
           y_pred[i] = (w_1 * x_i1) + (w_2 * x_i2) + ... + (w_p * x_ip) + b

     b. CALCULATE BASE ERRORS (Ordinary Least Squares):
        FOR each data point i:
           error[i] = y_pred[i] - y_i
```

```
    c. UPDATE EACH WEIGHT USING SOFT-THRESHOLDING (L1 Regularization):
       FOR each feature weight w_j:

          # Calculate unconstrained gradient step (OLS update)
          gradient_j = (2 / N) * SUM(error[i] * x_ij)
          w_j_temp = w_j - (alpha * gradient_j)

          # Apply L1 Soft-Thresholding Penalty
          IF w_j_temp > (alpha * lambda):
             w_j = w_j_temp - (alpha * lambda)
          ELSE IF w_j_temp < -(alpha * lambda):
             w_j = w_j_temp + (alpha * lambda)
          ELSE:
             w_j = 0  # FORCE COEFFICIENT EXACTLY TO ZERO

    d. UPDATE INTERCEPT (Unpenalized):
       b_gradient = (2 / N) * SUM(error[i])
       b = b - (alpha * b_gradient)

3. PREDICT NEW DATA:
   FOR any new sample X_new:
      # Features whose weight w_j was forced to 0 contribute nothing
      Y_new = SUM(w_j * X_new_j) + b
   RETURN Y_new
```

```
1. TRAINING / LEARNING PHASE:

   a. CALCULATE CLASS PRIOR PROBABILITIES:
      FOR each class C:
         P(C) = (Count of training samples in class C) / (Total training samples)

   b. CALCULATE FEATURE LIKELIHOODS:
      FOR each class C:
         FOR each feature X_j:

            # For Categorical Features:
            P(X_j | C) = (Count of samples in class C where feature = X_j) / (Total samples in
class C)

            # For Continuous Features (using Gaussian assumption):
            Calculate mean (mu) and variance (sigma^2) of feature X_j within class C

2. PREDICTION PHASE (For a new sample X_new = [x_1, x_2, ..., x_p]):

   FOR each candidate class C:

      a. START WITH CLASS PRIOR:
         posterior_score[C] = P(C)
```

```
    b. MULTIPLY BY EACH FEATURE'S LIKELIHOOD (Naïve Independence Assumption):
      FOR each feature x_j in X_new:

        # Categorical: Look up pre-calculated probability
        likelihood = P(x_j | C)

        # Continuous: Plug x_j into Gaussian probability density function
        # likelihood = PDF(x_j, mu_C, sigma^2_C)

        posterior_score[C] = posterior_score[C] * likelihood

3. DECISION STEP:
  # Choose the class with the maximum posterior score
  Predicted_Class = Class with HIGHEST posterior_score[C]

  RETURN Predicted_Class
```

```
1. INITIALIZATION:
  Set weight vector (w) = [0, 0, ..., 0]  (one weight per feature)
  Set bias (b) = 0
  Set regularization parameter (C) = 1.0  (controls margin strictness)
  Set learning rate (alpha) = 0.001
  Set number of iterations = 1000
  Ensure target labels (y) are encoded as -1 or +1

2. TRAINING LOOP (Hinge Loss Gradient Descent):
  FOR iteration FROM 1 TO iterations:

    FOR each training point (x_i, y_i):

      a. CHECK MARGIN CONDITION:
        # Functional margin evaluates if point is correctly classified
        # and outside the boundary region: y_i * (w · x_i + b) >= 1
        margin_score = y_i * (DOT_PRODUCT(w, x_i) + b)

      b. IF POINT IS OUTSIDE MARGIN (margin_score >= 1):
        # Correctly classified with safe margin -> apply only L2 weight decay
        w_gradient = w
        b_gradient = 0

      c. ELSE (margin_score < 1):
        # Misclassified or inside the margin -> penalize based on error
        w_gradient = w - (C * y_i * x_i)
        b_gradient = - (C * y_i)

      d. UPDATE WEIGHTS & BIAS:
        w = w - (alpha * w_gradient)
        b = b - (alpha * b_gradient)
```

```
3. PREDICTION PHASE (For a new sample X_new):
  score = DOT_PRODUCT(w, X_new) + b

  IF score >= 0:
    RETURN +1
  ELSE:
    RETURN -1
```

```
1. INITIALIZATION:
  Select Kernel Function K(x_i, x_j)  # e.g., RBF Kernel: exp(-gamma * ||x_i - x_j||^2)
  Set Dual Coefficients (alpha_1, alpha_2, ..., alpha_N) = 0
  Set bias (b) = 0
  Set regularization parameter (C) = 1.0
  Ensure target labels (y) are encoded as -1 or +1

2. PRE-COMPUTE KERNEL MATRIX (Gram Matrix):
  FOR i FROM 1 TO N:
    FOR j FROM 1 TO N:
      Kernel_Matrix[i][j] = K(x_i, x_j)  # Measure similarity between all pairs in input space

3. TRAINING LOOP (Sequential Minimal Optimization / Dual Formulation):
  FOR iteration FROM 1 TO iterations:

    FOR i FROM 1 TO N:

      a. PREDICT CURRENT SCORE FOR POINT i (Using Dual Form):
        # Score = SUM(alpha_j * y_j * K(x_j, x_i)) + b
        score_i = SUM(alpha[j] * y[j] * Kernel_Matrix[j][i]) + b

      b. CHECK KKT CONDITIONS (Margin Violations):
        error_i = score_i - y[i]

        # If point i violates constraints, pick a second point j and optimize (alpha_i, alpha_j)
pair:
        IF (y[i] * error_i < -tol AND alpha[i] < C) OR (y[i] * error_i > tol AND alpha[i] > 0):

          Select point j randomly or via heuristic
          Calculate optimal step update for (alpha[i], alpha[j])

          c. CLIP ALPHAS TO BOX CONSTRAINTS:
            # Enforce bounds: 0 <= alpha <= C
            alpha[i] = CLIP(alpha[i]_new, 0, C)
            alpha[j] = CLIP(alpha[j]_new, 0, C)

    d. UPDATE BIAS (b):
      Recompute bias b using support vectors (where 0 < alpha_k < C)

4. FILTER SUPPORT VECTORS:
  # Retain only points with non-zero alpha values
```

```
  Support_Vectors = { x_i | alpha[i] > 0 }
  Support_Alphas  = { alpha[i] | alpha[i] > 0 }
  Support_Labels  = { y[i] | alpha[i] > 0 }

5. PREDICTION PHASE (For a new sample X_new):
  # Calculate non-linear decision function using only Support Vectors
  score = SUM( Support_Alphas[k] * Support_Labels[k] * K(Support_Vectors[k], X_new) ) + b

  IF score >= 0:
    RETURN +1
  ELSE:
    RETURN -1
```

```
1. PRE-PROCESSING / STORAGE PHASE:
  # K-NN is a "Lazy Learner" (no explicit parameter training/optimization step)
  Store all training features (X_train) and their target labels/values (Y_train)
  Set hyperparameter K (number of neighbors)
  Set distance metric (e.g., Euclidean distance)

2. PREDICTION PHASE (For a single new sample X_new):

  a. CALCULATE DISTANCES:
    FOR each sample x_i in X_train:
      # Euclidean Distance: sqrt( SUM( (X_new_j - x_ij)^2 ) )
      distance[i] = DISTANCE(X_new, x_i)

  b. FIND NEAREST NEIGHBORS:
    Sort all training samples by distance[i] in ascending order
    Select top K training samples with the smallest distances:
      K_neighbors_indices = Indices of the K smallest distances

  c. AGGREGATE NEIGHBOR TARGETS:

    # IF CLASSIFICATION TASK (Discrete Labels):
      # Predict the majority class among the K neighbors (Mode)
      neighbor_labels = Y_train[K_neighbors_indices]
      Predicted_Output = MOST_COMMON(neighbor_labels)

    # IF REGRESSION TASK (Continuous Values):
      # Predict the average target value among the K neighbors (Mean)
      neighbor_values = Y_train[K_neighbors_indices]

      # Optional: Uniform vs Distance-Weighted Average
      IF distance_weighting IS True:
        weights = 1 / (distances[K_neighbors_indices] + epsilon)
        Predicted_Output = SUM(weights * neighbor_values) / SUM(weights)
      ELSE:
        Predicted_Output = MEAN(neighbor_values)
```

```
3. RETURN PREDICTION:
  RETURN Predicted_Output
```

```
1. TRAINING PHASE (Recursive Tree Construction):

  FUNCTION BuildTree(dataset, depth):

    # Step a: Check Termination / Stopping Criteria
    IF all samples in dataset have the same target class OR
      current depth >= max_depth OR
      number of samples < min_samples_split:
        RETURN LeafNode(prediction = Most Common Class OR Average Target Value)

    # Step b: Find Best Feature and Threshold to Split Data
    best_score = Infinity (or -Infinity)
    best_split = None

    FOR each feature f in dataset:
      FOR each unique value threshold v of feature f:

        Split dataset into Left_Group (f <= v) and Right_Group (f > v)

        # Measure split quality (e.g., Gini Impurity, Entropy, or Variance Reduction)
        split_score = CALCULATE_IMPURITY(Left_Group, Right_Group)

        IF split_score is better than best_score:
          best_score = split_score
          best_split = {feature: f, threshold: v, left: Left_Group, right: Right_Group}

    # Step c: Recursively Build Left and Right Subtrees
    left_child = BuildTree(best_split.left, depth + 1)
    right_child = BuildTree(best_split.right, depth + 1)

    RETURN InternalNode(feature = best_split.feature,
                        threshold = best_split.threshold,
                        left = left_child,
                        right = right_child)


2. PREDICTION PHASE (Traversing the Tree for X_new):

  FUNCTION Predict(node, X_new):

    # If we reached a leaf, return its stored output
    IF node is LeafNode:
      RETURN node.prediction

    # Route sample left or right based on decision rule
    IF X_new[node.feature] <= node.threshold:
```

```
      RETURN Predict(node.left, X_new)
    ELSE:
      RETURN Predict(node.right, X_new)
```

```
1. TRAINING PHASE (Building B Decision Trees in Parallel):

  Set number of trees = B
  Set subset feature count = m (typically sqrt(total_features) for classification, total_features/3
for regression)

  FOR b FROM 1 TO B:

    a. BOOTSTRAP SAMPLING (Data Randomness):
      Create a Bootstrap Sample of size N from original dataset by sampling with
replacement.
      (Unused ~36.8% of data forms the Out-of-Bag (OOB) validation set for tree b).

    b. RECURSIVELY BUILD DECORRELATED TREE:
      FUNCTION BuildRandomTree(bootstrap_dataset, depth):

        IF stopping_criteria_met:
          RETURN LeafNode(prediction)

        # Feature Randomness Step:
        Randomly select a subset of m features from all available features P.

        Evaluate splits ONLY among these m randomly selected features.
        Find best split {feature_f, threshold_v} that minimizes impurity (Gini/Variance).

        Split bootstrap_dataset into Left and Right groups.

        left_child = BuildRandomTree(Left_Group, depth + 1)
        right_child = BuildRandomTree(Right_Group, depth + 1)

        RETURN InternalNode(feature = feature_f, threshold = threshold_v, left = left_child,
right = right_child)

    Save Tree[b] to the Forest collection.


2. PREDICTION PHASE (Aggregating Forest Output for X_new):

  # Collect predictions from every tree in parallel
  FOR b FROM 1 TO B:
    tree_predictions[b] = Traverse Tree[b] for X_new

  IF CLASSIFICATION TASK:
    # Majority Vote (Mode) across all trees
    RETURN MOST_COMMON(tree_predictions)
```

```
IF REGRESSION TASK:
  # Average (Mean) across all trees
  RETURN MEAN(tree_predictions)
```